```
eip: 0000
title: ERC-3643 Subset Fungible Interface
description: ERC-20 interface defining the core ERC-3643-compatible compliance
subset
author: Ryan Sauge (@rya-sge)
discussions-to: https://ethereum-magicians.org/
status: Draft
type: Standards Track
category: ERC
created: 2026-02-12
requires: 20
```

# Abstract

This ERC defines a minimal, ERC-3643-inspired interface for fungible tokens that need compliance controls without prescribing a specific identity framework or access-control model.

The interface focuses on practical core primitives: transfer eligibility checks, pause/unpause, account and balance freezing, forced transfer, mint/burn, and version reporting.

# Motivation

The main goal is to stay maximally aligned with ERC-3643 naming and behavior for core compliance operations, while reducing mandatory surface area.

ERC-3643 includes broader functionality (such as identity-registry coupling and agent-oriented operational patterns) that is not required in all deployments.

Many issuers and protocols need a smaller interoperable surface:

- token issuers need operational controls (pause, freeze, forced transfer, mint/burn),
- integrators need a standardized control surface for lifecycle and enforcement operations,
- tooling and audits need implementation identification (`version`).

This ERC standardizes those core features while remaining:

- **identity-agnostic** (no mandatory on-chain identity architecture),
- **access-control-agnostic** (no mandated role/ownership scheme).

This ERC does not introduce new functions or events outside the ERC-3643 core subset used here. As a result, ERC-3643-compliant fungible token contracts are compatible with this interface.

Issuers that do not need ERC-3643 compatibility can also consider ERC-7943 as an alternative compliance-oriented interface family.

# Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

## Interface

```
interface IERC3643SubsetFungible {
    // Events
    event AddressFrozen(address indexed userAddress, bool indexed isFrozen,
address indexed owner);
    event TokensFrozen(address indexed userAddress, uint256 amount);
    event TokensUnfrozen(address indexed userAddress, uint256 amount);
    event Paused(address userAddress);
    event Unpaused(address userAddress);

    // View functions
    function version() external view returns (string memory version_);
    function paused() external view returns (bool);
    function isFrozen(address userAddress) external view returns (bool
isFrozen_);
    function getFrozenTokens(address userAddress) external view returns
(uint256 frozenTokens_);

    // State-changing compliance functions
    function pause() external;
    function unpause() external;
    function setAddressFrozen(address userAddress, bool freeze) external;
    function freezePartialTokens(address userAddress, uint256 amount) external;
    function unfreezePartialTokens(address userAddress, uint256 amount)
external;
    function forcedTransfer(address from, address to, uint256 value) external
returns (bool success_);
    function mint(address to, uint256 amount) external;
    function burn(address userAddress, uint256 amount) external;
}

interface IERC3643SubsetFungibleERC165 is IERC3643SubsetFungible, IERC165 {}
```

## Required Behavior

1. **Base standard**
   - Implementations MUST be compatible with ERC-20.
2. **Pause behavior**
   - `paused()` MUST reflect whether standard user transfers are currently blocked.
   - While paused, standard holder-initiated transfers (`transfer`, `transferFrom`) MUST revert.
3. **Freezing behavior**
   - `isFrozen(account)` MUST expose account-level freeze status.
   - `getFrozenTokens(account)` MUST expose frozen token amount.
4. **Enforcement and supply actions**
   - `forcedTransfer`, `mint`, `burn`, `setAddressFrozen`, `freezePartialTokens`, and `unfreezePartialTokens` MUST be restricted to authorized actors.
   - This ERC does not mandate the authorization mechanism.
5. **Version reporting**
   - `version()` MUST return the implementation version string.

    - Returned values SHOULD follow a SemVer-like format `MAJOR.MINOR.PATCH`.

### ERC-165 (Optional)

Implementations SHOULD support ERC-165 interface discovery for this interface.

- If implemented, `supportsInterface(type(IERC3643SubsetFungible).interfaceId)` SHOULD return `true`.
- The interface id for `IERC3643SubsetFungible` is `0xca989a3a`.

### Non-Goals

This ERC intentionally does not mandate:

- a specific on-chain identity system,
- a specific compliance engine architecture,
- a specific access-control standard (owner, roles, multisig, governance, etc.),
- mandatory batch operations.

Batch operations are intentionally excluded from this interface because batching can already be achieved through other mechanisms, such as ERC-6357 multicall patterns or smart wallet/account-abstraction execution flows.

## Rationale

- **Lean interoperability**: Keeps only commonly needed primitives for regulated fungible tokens.
- **Maximum ERC-3643 alignment**: Preserves ERC-3643-style function/event semantics for the core subset so integrators can reuse existing patterns.
- **Implementation freedom**: Leaves identity and authorization architecture to local regulatory and operational needs.

## Backwards Compatibility

This ERC is additive relative to ERC-20. Existing ERC-20 integrations remain valid, while advanced integrations can use this interface for compliance-aware behavior.

This ERC introduces no additional function or event requirements beyond the selected ERC-3643 subset. Therefore, ERC-3643-compliant fungible token implementations are compatible with this interface.

## Security Considerations

- Sensitive state-changing functions (`pause`, `unpause`, freeze, forced transfer, mint, burn) require robust authorization controls.
- Operators and integrators SHOULD maintain clear operational procedures for emergency controls (pause/freeze/enforcement).

## Copyright